

Lab 1: Bootloader & Physical Memory Management

Due: January 31, 2025.

Bootloader

Question A:

1. What is the first instruction that the processor executes in 32-bit mode? What exactly causes the switch from 16- to 32-bit mode? (Hint: use the 80836 Programmer's Reference Manual to investigate what each instruction does; do not simply take source code names/comments at face value.)

```

101  /* Load the bootstrap GDT
102  */
103  switch_prot:
104      lgdt    gdt_desc
105      movl    %cr0, %eax
106      orl    $CR0_PE_ON, %eax
107      movl    %eax, %cr0
108      /*
109       * switch to the protected mode
110       */
111      ljmp    $PROT_MODE_CSEG, $protcseg
112
113      .code32
114  protcseg:
115      movw    $PROT_MODE_DSEG, %ax
116      movw    %ax, %ds
117      movw    %ax, %es

```

After “ljmp \$PROT_MODE_CSEG, \$protcseg” (long jump) is done and CPU switches from 16- to 32-bit mode (requires same privilege levels), where \$PROT_MODE_CSEG (in our case 0x8) is the code segment (symbols location) and \$protcseg is the offset-address to jump in (32-bit mode entry point), the first command executed is “movw \$PROT_MODE_DSEG, %ax” that loads the 32-bit data segment into various %ds, %es, %fs, %gs, %ss registers needed to be defined, so that CPU will further look, extra data, and stack information in a larger 32-bit VA Space.

2. What is the last instruction of the boot loader executed, and what is the first instruction of the kernel it just loaded?

```

0x8f7e <boot1main+142>      call    0x8f8b <putc_tty>
B+ 0x8f73 <boot1main+147>      pop     %rax
0x8f74 <boot1main+148>      lea     0x60(%rbx),%eax
0x8f7a <boot1main+154>      pop     %rdx
0x8f7b <boot1main+155>      push    %rax
0x8f7c <boot1main+156>      push    %rsi
> 0x8f7d <boot1main+157>      call    0x8f9b <exec_kernel>
0x8f82 <boot1main+162>      lea     -0x2dd(%rbx),%eax
0x8f88 <boot1main+168>      mov     %eax,(%rsp)
0x8f8b <boot1main+171>      call    0x8be9 <panic>
0x8f90 <boot1main+176>      lea     -0x8(%rbp),%esp
0x8f93 <boot1main+179>      pop     %rbx
0x8f94 <boot1main+180>      pop     %rsi
0x8f95 <boot1main+181>      pop     %rbp
0x8f96 <boot1main+182>      ret
0x8f97 <__x86.get_pc_thunk.bx> mov     (%rsp),%ebx
0x8f9a <__x86.get_pc_thunk.bx+3> ret
0x8f9b <exec_kernel>        cli
0x8f9c <exec_kernel+1>      mov     $0x2badb002,%eax

```

This answer can vary depending on the success of the execution. Although, normally the kernel should not fail and end up running, there also might be a case where the symbols for the kernel entry point function are not found, with panic function to be called and the last “ret” boot command to be executed. However, in the normal case the last command would be “call 0x8f9b <exec_kernel>” located right underneath (look after the PC is adjusted at 0x8f97 <__x86.get_pc_thunk.bx>). It then jumps to the <exec_kernel> whose first command is “cli” cleaning the interrupt flags; disables interrupts. This is essential because the kernel needs to gain full control of the system and prevent interruptions during critical initialization.

3. Where is the first instruction of the kernel loaded in memory? (Hint: this should be a memory address.)

```

note Thread 1.1 In: exec_kernel
symbols from "/home/accts/am3785/workspace/pset-anton-and-oliver/obj/kern/kernel".
note target using gdb-specific protocol:
`/home/accts/am3785/workspace/pset-anton-and-oliver/obj/kern/kernel', file type elf32-i386
Entry point: 0x102944
0x0000000000100000 - 0x00000000001032a1 is .text
0x0000000000104000 - 0x00000000001046e8 is .rodata
0x00000000001046e8 - 0x0000000000105860 is .eh_frame
0x0000000000107000 - 0x000000000010700c is .got.plt
0x0000000000107020 - 0x0000000000107548 is .data
0x0000000000108000 - 0x000000000014ec30 is .bss
0x0000000000007c00 - 0x0000000000007c83 is .text in /home/accts/am3785/workspace/pset-anton-and-oliver/obj/kern/kernel
0x0000000000007e00 - 0x0000000000008fab is .text in /home/accts/am3785/workspace/pset-anton-and-oliver/obj/kern/kernel
0x0000000000008fab - 0x0000000000009098 is .rodata in /home/accts/am3785/workspace/pset-anton-and-oliver/obj/kern/kernel
0x0000000000009098 - 0x0000000000009340 is .eh_frame in /home/accts/am3785/workspace/pset-anton-and-oliver/obj/kern/kernel
0x0000000000009340 - 0x000000000000934c is .got.plt in /home/accts/am3785/workspace/pset-anton-and-oliver/obj/kern/kernel
0x0000000000009360 - 0x0000000000009400 is .data in /home/accts/am3785/workspace/pset-anton-and-oliver/obj/kern/kernel
0x0000000000009400 - 0x000000000000942c is .bss in /home/accts/am3785/workspace/pset-anton-and-oliver/obj/kern/kernel

```

So as previously mentioned, when the bootloader finishes loading the kernel and switches to protected mode, it jumps to this entry point (or similar). In our case this is 0x00102944 (32bit). It is usually marked as a <start> region.

Exercise 3. Trace through the first few instructions of the boot loader again and identify the first instruction that would “break” or otherwise do the wrong thing if you were to get the boot loader’s link address wrong. Then change the link address in `boot/boot0/Makefile.inc` to something wrong, execute `make clean`, recompile the lab with `make`, and trace into the bootloader again to see what happens. Don’t forget to change the link address back and execute `make clean` again afterward!

```

58  */
59  load boot1:
60  /* read disk */
61  pushl $0x0      # set the start
62  pushl $0x1      # LBA address
63  pushw %es       # set the buffer address
64  pushw $B00T1    # to 0x7e00
65  pushw $62       # set the number of sectors to be read
66  pushw $0x10     # set the size of DAP
67  movw %sp, %si   # set the DAP address
68  movw $0x4200, %ax # ah = 0x42, al = 0x00
69  int $0x13       # read sectors
70  jc load_fail    # encounter errors
71
72  jmp B00T1       # jump to boot1
73
74  load_fail:

```

Although GDB does not allow me to reach the faulting line, if the boot1 loader's link address is set incorrectly, the first issue we would assume likely occur at “int \$0x13,” command after all the configurations such as LBA, buffer addresses, size of DAP etc have been pushed to the stack. Boot0 invoked by BIOS calls interrupt 13 to read the sector from the disk, but because linking is pointing to invalid code or data or non-existent address, this jump could lead to variety of problems starting faulting silently, some undefined behavior, or triple faults (if protection is not implemented) with rebooting, depending what CPU actually finds. Analogically, if the boot0 loader’s link is invalid, we will not be even able to see the “Start boot0 ...” message, as it likely will fail immediately at the “int %10”. We see that it happens during the `<set_video_mode>` execution, which potentially means that incorrect location causes some memory corruption...

Question B. How does the boot loader decide how many sectors it must read in order to fetch the entire kernel from disk? Where does it find this information?

The boot loader determines how many sectors to read by examining the ELF header. The program header information, which contains the location and size of the segments to load, is accessed via the `e_phoff` field, which points to the offset of the program header in the ELF file. The loader uses this information to further decide `ph` and `eph` to calculate how many sectors are required to load the kernel into memory. In particular one would just iterate over and read every section with its entries in the table; each entry gives information such as the section name, the section size, and so on. Look at the example below to understand how mCertiKOS does it.

```

51     if (ELFHDR->e_magic != ELF_MAGIC)
52         panic("Kernel is not a valid elf.");
53
54     //load each program segment (ignores ph.flags)
55     ph = (proghdr *) ((uint8_t *) ELFHDR + ELFHDR->e_phoff);
56     eph = ph + ELFHDR->e_phnum;
57
58     for (; ph < eph; ph++) {
59         readsection(ph->p_va, ph->p_memsz, ph->p_offset, dkernel);
60     }
61
62     return (ELFHDR->e_entry & 0xFFFFF);
63 }

```

Exercise 4. We can examine memory using GDB's `x` command. The [GDB manual Links to an external site.](#) has full details, but for now, it is enough to know that the command `x/Nx ADDR` prints `N` words of memory at `ADDR`. (Note that both `x`s in the command are lowercase.) *Warning:* The size of a word is not a universal standard. In GNU assembly, a word is two bytes (the 'w' in `xorw`, which stands for word, means 2 bytes).

```

The target architecture is set to "i386:x86-64".

Breakpoint 1, exec_kernel () at boot/boot1/exec_kernel.S:6
(gdb) x/10x *0x89b
0x0: 0xf000ff53 0xf000ff53 0xf000e2c3 0xf000ff53
0x10: 0xf000ff53 0xf000ff54 0xf000ff53 0xf000ff53
0x20: 0xf000fea5 0xf000e987
(gdb)

```

Physical Memory Management

```

Testing the MATIntro layer...
test 1 passed.
test 2 passed.
test 3 passed.
All tests passed.

Testing the MATInit layer...
test 1 passed.
All tests passed.

Testing the MATOp layer...
test 1 passed.
All tests passed.

```